

Тема 5 Обзор SQL

Существует две формы языка SQL:

Интерактивный SQL - используется для функционирования непосредственно в базе данных, чтобы производить вывод информации для использования ее пользователем.

Вложенный SQL состоит из команд SQL помещенных внутри программ, которые обычно написаны на некотором другом языке.

Функциональные категории команд языка SQL:

DDL – состоит из команд которые создают объекты (таблицы, индексы, представления и так далее) в базе данных

DML – это набор команд, которые определяют какие значения представлены в таблицах в любой момент времени, а также позволяют помещать, изменять и удалять данные из таблиц БД;

DCL (Data Control Language, Язык Управления Данными) – состоит из средств, которые управляют выполнением транзакций, а также определяют разрешить ли пользователю выполнять определенные действия или нет.

Операторы определения данных DDL приведены в таблице ниже:

Оператор	Смысл	Действие
CREATE TABLE	Создать таблицу	Создает новую таблицу в БД
DROP TABLE	Удалить таблицу	Удаляет таблицу из БД
ALTER TABLE	Изменить таблицу	Изменяет структуру существующей таблицы или ограничения целостности, задаваемые для данной таблицы
CREATE VIEW	Создать представление	Создает виртуальную таблицу, соответствующую некоторому SQL-запросу
ALTER VIEW	Изменить представление	Изменяет ранее созданное представление
DROP VIEW	Удалить представление	Удаляет ранее созданное представление
CREATE INDEX	Создать индекс	Создает индекс для некоторой таблицы для обеспечения быстрого доступа по атрибутам, входящим в индекс
DROP INDEX	Удалить индекс	Удаляет ранее созданный индекс

Операторы манипулирования данными DML приведены в таблице ниже:

Оператор	Смысл	Действие
DELETE	Удалить строки	Удаляет одну или несколько строк, соответствующих условиям фильтрации, из базовой таблицы. Применение оператора согласуется с принципами поддержки целостности, поэтому этот оператор не всегда может быть выполнен корректно, даже если синтаксически он записан правильно
INSERT	Вставить строку	Вставляет одну строку в базовую таблицу. Допустимы модификации оператора, при которых сразу несколько строк могут быть перенесены из одной таблицы или запроса в базовую таблицу
UPDATE	Обновить строку	Обновляет значения одного или нескольких столбцов в одной или нескольких строках, соответствующих условиям фильтрации

Язык запросов Data Query Language (DQL)

Оператор	Смысл	Действие
SELECT	Выбрать строки	Оператор, заменяющий все операторы реляционной алгебры и позволяющий сформировать результирующее отношение, соответствующее запросу

Средства управления транзакциями

Оператор	Смысл	Действие
COMMIT	Завершить транзакцию	Завершить комплексную взаимосвязанную обработку информации, объединенную в транзакцию
ROLLBACK	Откатить транзакцию	Отменить изменения, проведенные в ходе выполнения транзакции
SAVEPOINT	Сохранить промежуточную точку выполнения транзакции	Сохранить промежуточное состояние БД, пометить его для того, чтобы можно было в дальнейшем к нему вернуться

Средства администрирования данных

Оператор	Смысл	Действие
ALTER DATABASE	Изменить БД	Изменить набор основных объектов в базе данных, ограничений, касающихся всей базы данных
ALTER DBAREA	Изменить область хранения БД	Изменить ранее созданную область хранения

Операторы управления данными DCL

Оператор	Смысл	Действие
ALTER PASSWORD	Изменить пароль	Изменить пароль для всей базы данных
CREATE DATABASE	Создать БД	Создать новую базу данных, определив основные параметры для нее
CREATE DBAREA	Создать область хранения	Создать новую область хранения и сделать ее доступной для размещения данных
DROP DATABASE	Удалить БД	Удалить существующую базу данных (только в том случае, когда вы имеете право выполнить это действие)
DROP DBAREA	Удалить область хранения БД	Удалить существующую область хранения (если в ней на настоящий момент не располагаются активные данные)
GRANT	Предоставить права	Предоставить права доступа на ряд действий над некоторым объектом БД
REVOKE	Лишить прав	Лишить прав доступа к некоторому объекту или некоторым действиям над объектом

Основные возможности SQL:

- предложения определения данных;
- запросы на выбор данных;
- предложения модификации данных;
- предложения управления данными

Кроме того, он предоставляет возможность выполнять в этих предложениях:

- арифметические вычисления (включая разнообразные функциональные преобразования), обработку текстовых строк и выполнение операций сравнения значений арифметических выражений и текстов;
- упорядочение строк и (или) столбцов при выводе содержимого таблиц на печать или экран дисплея;
- создание представлений (виртуальных таблиц), позволяющих пользователям иметь свой взгляд на данные без увеличения их объема в базе данных;
- запоминание выводимого по запросу содержимого таблицы, нескольких таблиц или представления в другой таблице (реляционная операция присваивания).
- группирование (агрегатирование) данных и применение к этим группам таких операций, как среднее, сумма, максимум, минимум, число элементов и т.п.

Достоинства языка SQL

- стандартность языка SQL;
- возможность переноса с одной вычислительной системы на другую;
- реляционная основа языка;
- возможность создания интерактивных запросов;
- возможность программного доступа к БД;
- обеспечение различного представления данных;
- возможность динамического изменения и расширения структуры БД;
- поддержка архитектуры клиент/сервер.

Типы данных в SQL

Типы данных, распознаваемые стандартом SQL ANSI, состоят из символов и различных типов чисел, которые могут классифицироваться как *точные* числа и *приблизительные* числа.

Точные числовые типы — это номера, с десятичной точкой или без десятичной точки.

Приблизительные числовые типы — это номера в показательной (экспоненциальной по основанию 10) записи.

Для всех прочих типов, отличия слишком малы чтобы их как-то классифицировать.

Иногда типы данных используют аргумент, который я называю *размером аргумента*, чей точный формат и значение меняется в зависимости от конкретного типа.

Значения по умолчанию обеспечены для всех типов, если размер аргумента отсутствует.

Может быть любым из следующих:

INTEGER (ЦЕЛОЕ ЧИСЛО),

CHARACTER (СИМВОЛЬНОЕ),
 DECIMAL (ДЕСЯТИЧНОЕ),
 NUMERIC (ЧИСЛОВОЕ),
 SMALLINT (НАИМЕНЬШЕЕ ЦЕЛОЕ)
 FLOAT (С ПЛАВАЮЩЕЙ ТОЧКОЙ)
 REAL (РЕАЛЬНОЕ),
 DOUBLE PRECISION (УДВОЕННАЯ ТОЧНОСТЬ С ПЛАВАЮЩЕЙ ТОЧКОЙ),
 LONG * (ДЛИННОЕ *),
 VARCHAR * (ПЕРЕМЕННОЕ СИМВОЛЬНОЕ) *,
 DATE (ДАТА *),
 TIME (ВРЕМЯ)
 (* - указывает на нестандартный для SQL тип данных)

Для даты, дает возможность выполнять арифметические операции с датами. Например, вы можете добавлять число дней к дате и получать другую дату в программе, самостоятельно следящей за числом дней в месяцах, високосными годами и так далее. Даты могут также сравниваться; например фраза, дата А < дата В означает, что дата А предшествует дате В по времени.

Кроме даты, большое количество программ определяют специальный тип для *времени*, который может также быть представлен в ряде форматов, включая следующие:

Стандарт	Формат	Пример
Международная Организация По Стандартизации (ISO)	hh-mm-ss	21.04.37
Японский Индустриальный Стандарт (JIS)	hh-mm-ss	21.04.37
IBM Европейский Стандарт	hh-mm-ss	21.04.37
IBM USA Стандарт (USA)	hh.mm AM/P	9.04 PM

Время может добавляться или сравниваться точно также как *дата*, с коррекцией числа секунд в минутах или часах автоматически. Кроме того, специально встроенные константы, указывающие текущую дату или время (**CURDATE** или **CURTIME**) являются общими. Они похожи на константу **USER** (Пользователь), в которой их значение будет непрерывно меняться.

Команда создания таблицы

```

CREATE TABLE < table name > (
  { < column name > < data type > [ < size > ]
    [ < colconstr > ... ] } , ...
  [ < tabconstr > ] , ... );
  
```

где:

< table name > - имя таблицы создаваемой этой командой

< column name > - имя столбца таблицы

< data type > - тип данных, который может содержаться в столбце.

< size > - размер поля. Его значение зависит от <data type>

< colconstr > - ограничение столбца. Может быть любым из следующих:

NOT NULL (НЕ НУЛЕВОЙ),

UNIQUE (УНИКАЛЬНЫЙ),

PRIMARY KEY (ПЕРВИЧНЫЙ КЛЮЧ),

CHECK(<predicate>) (ПРОВЕРКА предиката),

DEFAULT = ПО УМОЛЧАНИЮ =<value expression> значимому выражению)

REFERENCES <table name> (ССЫЛКА НА имя таблицы

[(<column name> , ...)] [(имя столбца)])

< tabconstr > - ограничение таблицы.

Может быть любым из следующих:

UNIQUE (УНИКАЛЬНЫЙ),

PRIMARY KEY (ПЕРВИЧНЫЙ КЛЮЧ),

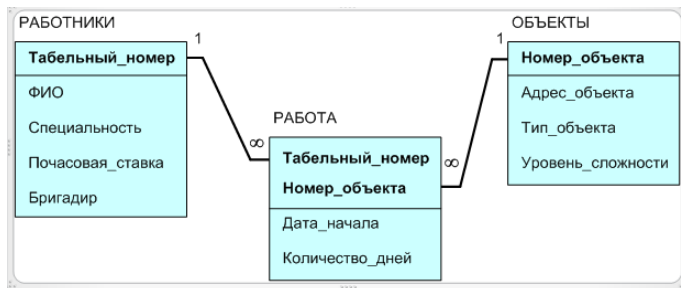
CHECK(<predicate>) (ПРОВЕРКА предиката),

FOREIGN KEY(<column name>) (ВНЕШНИЙ КЛЮЧ)

REFERENCES <table name> (ССЫЛКА НА имя таблицы

[(<column name>, ...)] [(имя столбца)]

Рассмотрим информационные сущности БД «Рембригада».



Создадим таблицы для БД «Рембригада».

1.

```
CREATE TABLE РАБОТНИКИ (
    ТАБЕЛЬНЫЙ_НОМЕР INTEGER NOT NULL PRIMARY KEY,
    ФИО CHARACTER (15),
    СПЕЦИАЛЬНОСТЬ CHARACTER (15),
    ПОЧАСОВАЯ_СТАВКА DECIMAL (6,1),
    БРИГАДИР INTEGER )
```

2.

```
CREATE TABLE ОБЪЕКТЫ (
    НОМЕР_ОБЪЕКТА INTEGER NOT NULL PRIMARY KEY,
    АДРЕС_ОБЪЕКТА CHARACTER (15),
    ТИП_ОБЪЕКТА CHARACTER (9) DEFAULT 'офис'
    CHECK (TYPE IN ('офис', 'склад', 'магазин', 'жилой дом' ) ),
    УРОВЕНЬ_СЛОЖНОСТИ INTEGER DEFAULT 1
    CHECK (УРОВЕНЬ_СЛОЖНОСТИ > 0 AND УРОВЕНЬ_СЛОЖНОСТИ < 5) )
```

3.

```
CREATE TABLE РАБОТА (
    ТАБЕЛЬНЫЙ_НОМЕР INTEGER NOT NULL,
    НОМЕР_ОБЪЕКТА INTEGER NOT NULL,
    ДАТА_НАЧАЛА DATE,
    КОЛИЧЕСТВО_ДНЕЙ INTEGER,
    PRIMARY KEY (ТАБЕЛЬНЫЙ_НОМЕР, НОМЕР_ОБЪЕКТА),
    FOREIGN KEY ТАБЕЛЬНЫЙ_НОМЕР REFERENCES РАБОТНИКИ
    ON DELETE CASCADE,
    FOREIGN KEY НОМЕР_ОБЪЕКТА REFERENCES ОБЪЕКТЫ
    ON DELETE CASCADE )
```

Изменение таблицы после того как она была создана осуществляется командой

ALTER TABLE < table name >

ADD < column name > < data type > < size > ;

- Столбец будет добавлен со значением NULL для всех строк таблицы.
- Новый столбец станет последним по порядку столбцом таблицы.
- Можно добавить сразу несколько новых столбцов, отделив их запятыми, в одной команде.
- Имеется возможность удалять или изменять столбцы.

Удаление таблицы осуществляется командой

DROP TABLE < table name >;

Чтобы иметь возможность удалить таблицу, пользователь должен иметь права владельца (т.е. быть создателем) этой таблицы.

Прежде, чем удалить таблицу из базы данных SQL потребует очистить ее от данных.

Ограничение значений данных в таблицах:

Ограничение столбца применяется только к индивидуальным столбцам.

Ограничение таблицы применяется к группам из одного и более столбцов.

Объявление ограничений

В таблице РАБОТНИКИ

ТАБЕЛЬНЫЙ_НОМЕР INTEGER NOT NULL PRIMARY KEY

В таблице ОБЪЕКТЫ

ТИП_ОБЪЕКТА CHARACTER (9) DEFAULT 'офис'

CHECK (TYPE IN ('офис', 'склад', 'магазин', 'жилой дом')),

УРОВЕНЬ_СЛОЖНОСТИ INTEGER DEFAULT 1

CHECK (УРОВЕНЬ_СЛОЖНОСТИ > 0 AND УРОВЕНЬ_СЛОЖНОСТИ < 5)

В таблице РАБОТА

PRIMARY KEY (ТАБЕЛЬНЫЙ_НОМЕР, НОМЕР_ОБЪЕКТА),

FOREIGN KEY ТАБЕЛЬНЫЙ_НОМЕР REFERENCES РАБОТНИКИ
ON DELETE CASCADE,

FOREIGN KEY НОМЕР_ОБЪЕКТА REFERENCES ОБЪЕКТЫ
ON DELETE CASCADE

Опции команды ON DELETE:

- CASCADE (распространить)
- SET NULL (установить пустое значение)
- SET DEFAULT (установить значение по умолчанию)

Создание запроса

Запрос — команда, которую вы даете вашей программе базы данных, и которая сообщает ей, чтобы она вывела определенную информацию из таблиц в память. Эта информация обычно посылается непосредственно на экран компьютера, которым вы пользуетесь, хотя, в большинстве случаев, ее можно также послать принтеру, сохранить в файле (как объект в памяти компьютера), или представить как вводную информацию для другой команды.

Запросы обычно рассматриваются как часть языка DML. Однако, так как запрос не меняет информацию в таблицах, а просто показывает ее пользователю, мы будем рассматривать запросы как самостоятельную категорию среди команд DML, которые производят действие, а не просто показывают содержание базы данных.

Все запросы в SQL состоят из одиночной команды — **SELECT (ВЫБОР)**.

Рассмотрим примеры создания запросов на учебной базе данных, состоящей из трех взаимосвязанных таблиц.

Таблица: **Продавцы**

SNUM	SNAME	CITY	COMM
1001	Peel	London	.12
1002	Serres	San Jose	.13
1004	Motika	London	.11
1007	Rifkin	Barcelona	.15
1003	Axelrod	New York	.10

Таблица: **Заказчики**

CNUM	CNAME	CITY	RATING	SNUM
2001	Hoffman	London	100	1001
2002	Giovanni	Rome	200	1003
2003	Liu	SanJose	200	1002
2004	Grass	Berlin	300	1002
2006	Clemens	London	100	1001
2008	Cisneros	SanJose	300	1007
2007	Pereira	Rome	100	1004

Таблица **Заказы**

ONUM	AMT	ODATE	CNUM	SNUM
3001	18.69	10/03/2022	2008	1007
3003	767.19	10/03/2022	2001	1001
3002	1900.10	10/03/2022	2007	1004
3005	5160.45	10/03/2022	2003	1002
3006	1098.16	10/03/2022	2008	1007
3009	1713.23	10/04/2022	2002	1003
3007	75.75	10/04/2022	2004	1002
3008	4723.00	10/05/2022	2006	1001
3010	1309.95	10/06/2022	2004	1002
3011	9891.88	10/06/2022	2006	1001

Рассмотрим структуру этих таблиц.

Здесь показаны столбцы Таблицы **Продавцы**

ПОЛЕ	СОДЕРЖАНИЕ
Snum	уникальный номер, назначенный каждому продавцу ("номер служащего")
Sname	имя продавца
City	расположение продавца (город)
Comm	комиссионные продавцов в десятичной форме

Таблица **Заказчики** содержит следующие столбцы:

ПОЛЕ	СОДЕРЖАНИЕ
Cnum	Уникальный номер, назначенный каждому заказчику
Cname	Имя заказчика
City	Расположение заказчика (город)
Rating	Код, указывающий уровень предпочтения данного заказчика перед другими. Более высокий номер указывают на большее предпочтение (рейтинг).
Snum	Номер продавца, назначенного этому заказчику (из таблицы Продавцов)

И имеются столбцы в Таблице **Заказы**:

ПОЛЕ	СОДЕРЖАНИЕ
Onum	уникальный номер, данный каждому приобретению
Amt	значение суммы приобретений
Odate	дата приобретения
Cnum	номер заказчика делающего приобретение (из таблицы Заказчиков)
Snum	номер продавца продающего приобретение (из таблицы Продавцов)

Команда SELECT

В самой простой форме, команда SELECT просто инструктирует базу данных, чтобы извлечь информацию из таблицы. Например, вы могли бы вывести таблицу Продавцов, напечатав следующее:

```
SELECT snum, sname, city, comm
FROM Salespeople;
```

Вывод для этого запроса показан на рисунке ниже.

snum	sname	city	comm
1001	Peel	London	0.12
1002	Serres	San Jose	0.13
1004	Motika	London	0.11
1007	Rifkin	Barcelona	0.15
1003	Axelrod	New York	0.10

Другими словами, эта команда просто выводит все данные из таблицы.

Если вы хотите видеть все столбцы таблицы, имеется необязательное сокращение, которое вы можете использовать. Звездочка (*) может применяться для вывода полного списка столбцов следующим образом:

```
SELECT *
FROM Salespeople;
```

Это приведет к тому же результату, что и наша предыдущая команда.

Команда SELECT способна извлечь строго определенную информацию из таблицы. Сначала, мы можем предоставить возможность увидеть только определенные столбцы таблицы. Это выполняется легко, простым исключением столбцов, которые вы не хотите видеть, из части команды SELECT. Например,

```
SELECT sname, comm
FROM Salespeople;
```

Переупорядочивание столбцов

Даже если столбцы таблицы, по определению, упорядочены, это не означает, что вы будете восстанавливать их в том же порядке. Конечно, оператор "SELECT *..." покажет все столбцы в их

естественном порядке, но если вы укажете столбцы отдельно, вы можете получить их в том порядке, в котором хотите.

Например, выведем таблицу Заказов, содержащую дату приобретения (odate), номер продавца (snum), номер Заказа (onum), и суммы приобретения (amt):

```
SELECT odate, snum, onum, amt
FROM Orders;
```

Вывод этого запроса показан на Рисунке ниже.

odate	snum	onum	amt
10/03/2022	1007	3001	18.69
10/03/2022	1001	3003	767.19
10/03/2022	1004	3002	1900.10
10/03/2022	1002	3005	5160.45
10/03/2022	1007	3006	1098.16
10/04/2022	1003	3009	1713.23
10/04/2022	1002	3007	75.75
10/05/2022	1001	3008	4723.00
10/06/2022	1002	3010	1309.95
10/06/2022	1001	3011	9891.88

Удаление избыточных данных

DISTINCT (ОТЛИЧИЕ) — аргумент, который обеспечивает способом устранять двойные значения из вашего предложения SELECT. Предположим что вы хотите знать, какие продавцы в настоящее время имеют свои Заказы в таблице Заказов. Под Заказом (здесь и далее) будет пониматься запись в таблицу Заказов, регистрирующую приобретения, сделанные в определенный день определенным заказчиком у определенного продавца на определенную сумму.

```
SELECT snum
FROM Orders;
```

snum
1007
1001
1004
1002
1007
1003
1002
1001
1002
1001

Для получения списка без дубликатов, для удобочитаемости, вы можете ввести следующее:

```
SELECT DISTINCT snum
FROM Orders;
```

Вывод для этого запроса показан на рисунке ниже

snum
1001
1002
1003
1004
1007

DISTINCT может указываться только один раз в данном предложении SELECT.

Квалифицированный выбор при использовании предложений

WHERE — предложение команды SELECT, которое позволяет вам устанавливать предикаты, условие которых может быть или верным или неверным для любой строки таблицы. Команда извлекает только те строки из таблицы, для которых такое утверждение верно. Предположим, вы хотите видеть имена и комиссионные всех продавцов в Лондоне. Вы можете ввести такую команду:

```
SELECT sname, comm
FROM Salespeople
WHERE city = 'London';
```

Например, можно выбрать всех заказчиков с рейтингом 100 следующим образом:

```
SELECT *  
FROM Customers  
WHERE rating = 100;
```

Одиночные кавычки не используются здесь потому, что оценка — это числовое поле.

Реляционные операторы

Реляционный оператор — математический символ, который указывает на определенный тип сравнения между двумя значениями. Вы уже видели, как используются равенства, такие как $2 + 3 = 5$ или `city = 'London'`. Но также имеются другие реляционные операторы. Предположим, что вы хотите видеть всех Продавцов с их комиссионными выше определенного значения. Вы можете использовать тип сравнения "больше чем" (`>`).

Реляционные операторы, которыми располагает SQL:

=	<i>Равно</i>
>	<i>Больше чем</i>
<	<i>Меньше чем</i>
>=	<i>Больше чем или равно</i>
<=	<i>Меньше чем или равно</i>
<>	<i>Не равно</i>

Например, вы хотите увидеть всех заказчиков с оценкой (rating) выше 200. Можно использовать реляционный оператор.

```
SELECT *  
FROM Customers  
WHERE rating > 200;
```

Если бы мы захотели увидеть еще и заказчиков с оценкой, равной 200, мы стали бы использовать предикат

```
rating >= 200
```

Булевы операторы

Выражения Буля — являются или верными, или неверными, подобно предикатам. Булевы операторы связывают одно или более верных/неверных значений и производят единственное верное/или/неверное значение. Стандартными операторами Буля, распознаваемыми в SQL, являются: AND, OR, и NOT.

Существуют другие, более сложные, операторы Буля (типа "исключенный или"), но они могут быть сформированы из этих трех простых операторов — AND, OR, NOT.

Связывая предикаты с операторами Буля, вы можете значительно увеличить их возможности. Например, надо вывести всех заказчиков в San Jose которые имеют рейтинг выше 200:

```
SELECT *  
FROM Customers  
WHERE city = 'San Jose'  
AND rating > 200;
```

Если вы используете OR, то получите всех заказчиков, которые находились в San Jose или (OR) которые имели оценку выше 200.

```
SELECT *  
FROM Customers  
WHERE city = 'San Jose' OR rating > 200;
```

NOT может использоваться для инвертирования значений Буля. Например, запрос с NOT:

```
SELECT *  
FROM Customers  
WHERE city = 'San Jose' OR NOT rating > 200;
```

Все записи, за исключением Grass, были выбраны. В каждой из других строк встретился один или другой или оба критериев. Обратите внимание, что оператор NOT должен предшествовать Булеву оператору, чье значение должно измениться, и не должен помещаться перед реляционным оператором.

Оператор IN

Оператор IN определяет набор значений, в который данное значение может или не может быть включено. В соответствии с нашей учебной базой Например, если вы хотите найти всех продавцов, которые размещены в Barcelona или в London, надо использовать следующий запрос.

```
SELECT *
FROM Salespeople
WHERE city = 'Barcelona' OR city = 'London';
```

Имеется и более простой способ получить ту же информацию:

```
SELECT *
FROM Salespeople
WHERE city IN ('Barcelona', 'London');
```

Оператор IN определяет набор значений с помощью имен членов набора, заключенных в круглые скобки и отделенных запятыми. Он затем проверяет различные значения указанного поля, пытаясь найти совпадение со значениями из набора. Если это случается, то предикат верен. Когда набор содержит значения номеров, а не символов, одиночные кавычки опускаются.

Пример, вывести всех заказчиков относящихся к продавцам, имеющих значения snum = 1001, 1007, и 1004.

```
SELECT *
FROM Customers
WHERE cnum IN (1001, 1007, 1004);
```

Оператор BETWEEN

Оператор BETWEEN похож на оператор IN. В отличие от определения по номерам из набора, как это делает IN, BETWEEN определяет диапазон, значения которого должны уменьшаться, что делает предикат верным. Вы должны ввести ключевое слово BETWEEN с начальным значением, ключевое AND и конечное значение. В отличие от IN, BETWEEN чувствителен к порядку, и первое значение в предложении должно быть первым в алфавитном или числовом порядке. Это применимо и к оператору LIKE (см. ниже). Следующий пример будет извлекать из таблицы Продавцов всех продавцов с комиссионными между .10 и .12

```
SELECT *
FROM Salespeople
WHERE comm BETWEEN .10 AND .12;
```

SQL не имеет поддержки исключения границ диапазона в выражении BETWEEN. Вы должны или определить ваши граничные значения так, чтобы включающая интерпретация была приемлема, или сделать что-нибудь типа этого:

```
SELECT *
FROM Salespeople
WHERE (comm BETWEEN .10, AND .12) AND NOT comm IN (.10, .12);
```

Также, подобно реляционным операторам, BETWEEN может работать с символьными полями. Это означает, что вы можете использовать BETWEEN, чтобы выбирать ряд значений из упорядоченных по алфавиту значений.

Этот запрос выбирает всех заказчиков, чьи имена попали в диапазон от 'A' до 'G':

```
SELECT *
FROM Customers
WHERE cname BETWEEN 'A' AND 'G';
```

Оператор LIKE

LIKE применим только к полям типа CHAR или VARCHAR, с которыми он используется, чтобы находить подстроки. То есть, он ищет поле символа, чтобы видеть, совпадает ли с условием часть его строки. В качестве условия он использует групповые символы, т.е. специальные символы, которые могут соответствовать чему-нибудь. Имеются два типа групповых символов используемых с LIKE:

символ подчеркивания () замещает любой одиночный символ. Например, 'b_t' будет соответствовать словам 'bat' или 'bit', но не будет соответствовать 'brat'.

знак процента (%) замещает последовательность любого числа символов (включая символы нуля). Например '%pr%' будет соответствовать словам 'put', 'posit', или 'opt', но не 'spite'.

Найдем всех заказчиков, чьи имена начинаются с G.

```
SELECT
FROM Customers
WHERE cname LIKE 'G%';
```

LIKE может быть удобен, если вы ищете имя или другое значение, и если вы не помните, как они точно пишутся. Предположим, что вы не уверены, как записано по буквам имя одного из продавцов Peal или Peel. Вы можете просто использовать ту часть, которую вы знаете, и групповые символы, чтобы находить все возможные пары:

```
SELECT *
FROM Salespeople
WHERE sname LIKE 'P _ _ l %';
```

Групповые символы подчеркивания, каждый из которых представляет один символ, добавляют только два символа к уже существующим 'P' и 'l', поэтому имя наподобие Prettel не может быть показано. Групповой символ '%' — в конце строки необходим в большинстве реализаций, если длина поля sname больше чем число символов в имени Peel, потому что некоторые другие значения sname — длиннее, чем четыре символа. В таком случае, значение поля sname, фактически сохраняемое как имя Peel, сопровождается рядом пробелов. Следовательно, символ 'l' не будет рассматриваться концом строки. Групповой символ '%' — просто соответствует этим пробелам. Это необязательно, если поля sname имеет тип VARCHAR.

В предикате LIKE вы можете определить любой одиночный символ как символ ESC. Символ ESC используется сразу перед процентом или подчеркиванием в предикате, и означает, что процент или подчеркивание будет интерпретироваться как символ, а не как групповой символ.

В примере выше и символ процента в начале, и символ процента в конце обрабатываются как групповые символы; только подчеркивание предоставлено само себе.

Как упомянуто выше, символ ESC может также использоваться самостоятельно. Другими словами, если вы будете искать столбец с вашим символом ESC, вы просто вводите его дважды. Во-первых, это будет означать, что символ ESC "берет следующий символ буквально как символ", и, во-вторых, что символ ESC самостоятелен.

Оператор NULL

Так как NULL указывает на отсутствие значения, вы не можете знать, каков будет результат любого сравнения с использованием NULL. Когда NULL сравнивается с любым значением, даже с другим таким же NULL, результат будет ни верным, ни неверным, он — неизвестен. Неизвестный Булев, вообще ведет себя также как неверная строка, которая, произведя неизвестное значение в предикате, не будет выбрана запросом — имейте в виду, что в то время как NOT (неверное) — равняется верно, NOT (неизвестное) — равняется неизвестно. Следовательно, выражение типа 'city = NULL' или 'city IN (NULL)' будет неизвестно, независимо от значения city. Часто вы должны делать различия между "неверно" и "неизвестно" — между строками, содержащими значения столбцов, которые не соответствуют условию предиката и которые содержат NULL в столбцах. По этой причине, SQL предоставляет специальный оператор IS, который используется с ключевым словом NULL, для размещения значения NULL. Найдем все записи в нашей таблице Заказчиков с NULL значениями в столбце city:

```
SELECT *
FROM Customers
WHERE city IS NULL;
```

Здесь не будет никакого вывода, потому что мы не имеем никаких значений NULL в наших типовых таблицах. Значения NULL — очень важны, и мы вернемся к ним позже.

Агрегатные функции

Запросы могут вычислять обобщенное групповое значение полей точно также как и значение одного поля. Это делает с помощью агрегатных функций. Агрегатные функции вычисляют одиночное значение для всей группы таблицы. Имеется список этих функций:

COUNT вычисляет номера строк или не NULL значения полей, которые выбрал запрос.

SUM вычисляет арифметическую сумму всех выбранных значений данного поля.

AVG вычисляет усреднение всех выбранных значений данного поля.

MAX вычисляет наибольшее из всех выбранных значений данного поля.

MIN вычисляет наименьшее из всех выбранных значений данного поля.

Агрегатные функции используются подобно именам полей в предложении SELECT запроса, но с одним исключением, они берут имена поля как аргументы. Только числовые поля могут использоваться с SUM и AVG.

С COUNT, MAX, и MIN, могут использоваться и числовые или символьные поля. Когда они используются с символьными полями, MAX и MIN будут транслировать их в эквивалент ASCII, который должен сообщать, что MIN будет означать первое, а MAX последнее значение в алфавитном порядке.

Пример, найти SUM всех покупок в таблице Заказов:

```
SELECT SUM (amt)
FROM Orders;
```

Функция COUNT несколько отличается от всех. Она считает число значений в данном столбце, или число строк в таблице. Когда она считает значения столбца, она используется с DISTINCT, чтобы вычислять число различных значений в данном поле. Например, надо вычислить число продавцов, описанных в таблице Заказов:

```
SELECT COUNT (DISTINCT snum)
FROM Orders;
```

Чтобы подсчитать общее число строк в таблице, используйте функцию COUNT со звездочкой вместо имени поля, например,:

```
SELECT COUNT (*)
FROM Customers
```

COUNT со звездочкой включает и NULL, и дубликаты, по этой причине не может быть использовано предложение DISTINCT.

Агрегатные функции могут также (в большинстве реализаций) использовать аргумент ALL, который помещается перед именем поля, подобно DISTINCT, но означает противоположное – включать дубликаты. ANSI технически не позволяет этого для COUNT, но многие реализации ослабляют это ограничение.

Различия между ALL и * когда они используются с COUNT:

ALL использует имя поля как аргумент.

ALL не может подсчитать значения NULL.

Следующая команда подсчитает (COUNT) число не NULL значений в поле rating в таблице Заказчиков (включая повторения):

```
SELECT COUNT (ALL rating)
FROM Customers;
```

До этого, вы использовали агрегатные функции с одиночными полями как аргументами. Можно также использовать агрегатные функции с аргументами, которые состоят из скалярных выражений, включающих одно или более полей. (Если вы это делаете, DISTINCT не разрешается.) Предположим, что таблица Заказов имеет еще один столбец, который хранит предыдущий неуплаченный баланс (поле blnc) для каждого заказчика. Надо найти текущий баланс, добавлением суммы приобретений к предыдущему балансу. Этот наибольший неуплаченный баланс можно получить следующим образом:

```
SELECT MAX (blnc + amt)
FROM Orders;
```

Для каждой строки таблицы этот запрос будет складывать blnc и amt для этого заказчика и выбирать максимальное значение.

Предложение GROUP BY

Предложение GROUP BY позволяет определять подмножество значений в поле в терминах другого поля, и применять функцию агрегата к подмножеству. Это дает возможность объединять поля и агрегатные функции в едином предложении SELECT. Например, вы хотите найти

наибольшую сумму приобретений, полученную каждым продавцом. Можно сделать отдельный запрос для каждого из них, выбрав MAX (amt) из таблицы Заказов для каждого значения поля snum:

```
SELECT snum, MAX (amt)
FROM Orders
GROUP BY snum;
```

GROUP BY применяет агрегатные функции независимо от серий групп, которые определяются с помощью значения поля в целом. В этом случае, каждая группа состоит из всех строк с тем же самым значением поля snum, и функция MAX применяется отдельно для каждой такой группы. Это значение поля, к которому применяется GROUP BY, имеет, по определению, только одно значение на группу вывода, также как это делает агрегатная функция. Можно использовать GROUP BY с многочисленными полями. Например, необходимо увидеть наибольшую сумму приобретений, получаемую каждым продавцом каждый день. Чтобы сделать это, вы должны сгруппировать таблицу Заказов по продавцам и датам, и применить функцию MAX к каждой такой группе:

```
SELECT snum, odate, MAX (amt)
FROM Orders
GROUP BY snum, odate;
```

В дни, когда текущий продавец не имел Заказов, не будут показаны в выводе.

Предложение HAVING

Предположим, что в предыдущем примере, вы хотели бы увидеть только максимальную сумму приобретений, значение которой выше \$3000.00. Вы не сможете использовать агрегатную функцию в предложении WHERE , потому что предикаты оцениваются в терминах одиночной строки, а агрегатные функции оцениваются в терминах группы строк. Это означает, что вы не сможете сделать что-нибудь подобно следующему:

```
SELECT snum, odate, MAX (amt)
FROM Orders
WHERE MAX (amt) > 3000.00
GROUP BY snum, odate;
```

Это будет отклонением от строгой интерпретации ANSI. Чтобы увидеть максимальную стоимость приобретений свыше \$3000.00, можно использовать предложение HAVING. Предложение HAVING определяет критерии, используемые для исключения определенных групп из вывода, также как предложение WHERE делает это для индивидуальных строк. Правильной командой будет следующая:

```
SELECT snum, odate, MAX (amt)
FROM Orders
GROUP BY snum, odate
HAVING MAX (amt) > 3000.00;
```

Аргументы в предложении HAVING следуют тем же самым правилам, что и в предложении SELECT, состоящем из команд использующих GROUP BY. Они должны иметь одно значение на группу вывода.

Предположим, что вы хотите выполнять простые числовые вычисления, чтобы затем помещать их в форму, больше соответствующую вашим потребностям. SQL позволяет помещать скалярные выражения и константы среди выбранных полей. Эти выражения могут дополнять или замещать поля в предложениях SELECT, и могут включать в себя одно или более выбранных полей.

Размещение текста в выводе запроса

Символ 'A', когда ничего не значит сам по себе – является константой, такой, например, как число 1. Вы можете вставлять константы в предложение SELECT запроса, включая и текст. Однако символьные константы, в отличие от числовых констант, не могут использоваться в выражениях. Например, представим комиссионные как проценты со знаком процента (%). Это даст возможность помещать в вывод такие единицы как символы и комментарии:

```
SELECT snum, sname, city, ' % ', comm * 100
FROM Salespeople;
```

Обратите внимание, что пробел перед процентом вставляется как часть строки. Необходимо помнить, что этот комментарий будет напечатан в каждой строке вывода, а не просто один раз для всей таблицы. Например, необходимо получить отчет, в котором указывается число Заказов, получаемых в течение каждого дня.

```
SELECT ' For ', odate, ', there are ',
COUNT (DISTINCT onum), 'orders.'
FROM Orders
GROUP BY odate;
```

=====				
odate				

For	10/03/2016	, there are	5	orders.
For	10/04/2016	, there are	2	orders.
For	10/05/2016	, there are	1	orders.
For	10/06/2016	, there are	2	orders.

```
=====
```

Упорядочение вывода полей

Таблицы – это неупорядоченные наборы данных. SQL использует команду ORDER BY, чтобы позволять упорядочивать вывод данных. Эта команда упорядочивает вывод запроса согласно значениям в том или ином количестве выбранных столбцов. Многочисленные столбцы упорядочиваются один внутри другого, также как с GROUP BY, и можно определять возрастание (ASC) или убывание (DESC) для каждого столбца. По умолчанию установлено – возрастание. Например, вывести из таблицы Заказов номера заказчика в порядке убывания:

```
SELECT *
FROM Orders
ORDER BY cnum DESC;
```

Можно упорядочивать таблицу по второму столбцу, например, по полю amt, внутри упорядочения поля cnum:

```
SELECT *
FROM Orders
ORDER BY cnum DESC, amt DESC;
```

Можно использовать ORDER BY таким же способом с любым числом столбцов. Во всех случаях, столбцы которые упорядочиваются должны быть указаны в выборе SELECT. Это – требование ANSI, которое в большинстве, но не всегда, предписано системе. Следующая команда, например, будет запрещена:

```
SELECT cname, city
FROM Customers
GROUP BY cnum;
```

Так как поле cnum не было выбранным полем, GROUP BY не сможет найти его, чтобы использовать для упорядочения вывода.

Вместо имен столбцов можно использовать их порядковые номера для указания поля, используемого в упорядочении вывода. Эти номера могут ссылаться не на порядок столбцов в таблице, а на их порядок в выводе. Другими словами, поле упомянутое в предложении SELECT первым, для ORDER BY – это поле 1, независимо от того каким по порядку оно стоит в таблице. Например, чтобы увидеть определенные поля таблицы Продавцов, упорядоченными в порядке убывания к наименьшему значению комиссионных, можно сделать следующий запрос:

```
SELECT sname, comm
FROM Salespeople
GROUP BY 2 DESC;
```

Одна из основных целей этой возможности ORDER BY – дать возможность использовать GROUP BY со столбцами вывода также как и со столбцами таблицы. Столбцы, производимые агрегатной функцией, константы или выражения в предложении SELECT запроса, абсолютно пригодны для использования с GROUP BY, если они ссылаются к ним с помощью номера. Например, подсчитать число заказов каждого продавца и вывести результат в порядке убывания:

```
SELECT snum, COUNT (DISTINCT onum)
FROM Orders
GROUP BY snum
ORDER BY 2 DESC;
```

В этом случае используется номер столбца, так как столбец вывода не имеет имени